

GOMOKU

Выполнила Демьянова Таисия Владимировна

1. Постановка задачи

«Реализовать консольную игру Gomoku на .NET, поле 15×15, два игрока, победа — 5 подряд по горизонтали/вертикали/диагонали».

В рамках учебного проекта преподавателем была поставлена задача разработать консольную игру «Gomoku».

Основные требования к реализации:

- Создать игровое поле размером **15×15 клеток**.
- Реализовать возможность игры двух участников (Hotseat).
- Предусмотреть проверку корректности хода.
- Обеспечить определение победителя при последовательности из пяти символов по горизонтали, вертикали или диагонали.
- Разработать консольный интерфейс с цветовым выделением символов игроков.
- Реализовать меню для запуска новой игры, продолжения, сохранения и загрузки состояния, а также отображения таблицы рекордов.

2. Описание игры

Гомоку — логическая игра для двух игроков. На квадратной доске игроки поочерёдно выставляют камни двух цветов. Выигрывает тот, кто первым построит непрерывный ряд из пяти камней своего цвета.

Правила и смысл игры:

- **Поле:** Игра ведётся на квадратном поле размером 15x15 линий.
- **Ход:** Игроки по очереди выставляют на свободные пункты доски фишки своего цвета. Первый ход делает игрок, играющий "крестиками" (или красными).
- **Цель:** Победителем становится тот, кто первым построит непрерывный ряд из ровно пяти своих камней.
- **Направления:** Ряд может быть построен по горизонтали, вертикали или диагонали.

3. Интерфейс пользователя

- Консольный вывод поля с координатами.
- Цветовое выделение символов (X — красный, O — синий).
- Сообщения о ходе, победе, ошибках.
- Главное меню с пунктами: новая игра, продолжение, сохранение, загрузка, настройки, рекорды, выход.

```
-----  
Player1           Red           0  
Player2           Blue          0  
-----  
  
=== Gomoku Menu ===  
1. Start New Game  
2. Continue Game  
3. Load Last Save  
4. Save Current Game  
5. Settings  
6. Top Scores  
7. Exit  
Enter your choice: █
```


4. Программная реализация

Приложение разработано с использованием объектно-ориентированного подхода на языке C#.

Технологический стек:

- **Язык программирования:** C# 12
- **Платформа:** .NET 9.0
- **Среда разработки:** Visual Studio 2022

Особенности реализации:

- Использование классов для декомпозиции задачи.
- Консольный интерфейс с цветовым выделением (ConsoleHelper).

Gomoku Game

```
-----
      00 01 02 03 04 05 06 07 08 09 10 11 12 13 14
00 . . . . . . . . . . . . . . .
01 . . . . . 0 X . . . . . . . . .
02 . . . . . X . . . . . . . . .
03 . . . 0 X X . . . . . . . . .
04 . . . X 0 X . . . . . . . . .
05 . . X 0 0 X . . . . . . . . .
06 . 0 . X 0 0 . . . . . . . . .
07 . . . . . . . . . . . . . . .
08 . . . . . . . . . . . . . . .
09 . . . . . . . . . . . . . . .
10 . . . . . . . . . . . . . . .
11 . . . . . . . . . . . . . . .
12 . . . . . . . . . . . . . . .
13 . . . . . . . . . . . . . . .
14 . . . . . . . . . . . . . . .
Player1 wins!
Press any key to continue...
```

5. Программная реализация: Состав

Проект представляет собой консольное приложение, состоящее из следующих классов:

Board

Отвечает за игровое поле, ходы и проверку победы.

Game

Управляет игровым циклом и игроками.

Menu

Реализует главное меню и навигацию.

Player

Описывает сущность игрока (имя, цвет, счёт).

ConsoleHelper

Обеспечивает цветной вывод и UI.

SaveManager

Сохранение и загрузка (JSON).

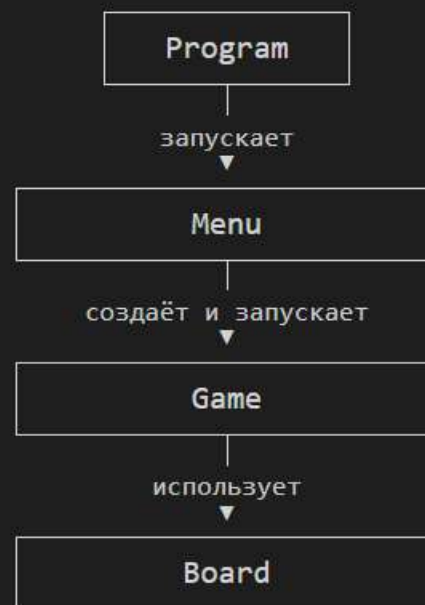
Converter

Сериализация массива `int[,]`.

Program

Точка входа.

6. Архитектура приложения



Game также использует:



Menu и Game используют:



Game использует:



Класс Board

Отвечает за игровое поле, выполнение ходов и проверку условий победы.

Ключевой момент: Векторная проверка победы

Вместо того чтобы писать 4 разных цикла для каждого направления, используется один универсальный метод `checkDirection`.

```
private bool checkDirection(int x, int y, int dx, int dy, int player) { // ... // Смещаемся по вектору (dx, dy) 5 раз int nx = x + i * dx; int ny = y + i * dy;
// ... }
```

Позволяет проверять любое направление (горизонталь, вертикаль, диагонали), просто меняя параметры смещения `dx` и `dy`. Это соответствует принципу DRY (Don't Repeat Yourself) и делает код компактным.

Валидация хода:

Метод `makeMove` защищает массив от выхода за пределы.

```
if (x < 0 || x >= _size || BoardArray[x, y] != 0) return false;
```

Гарантирует, что программа не «упадет» с ошибкой `IndexOutOfRangeException` при попытке поставить фишку вне поля или на занятую клетку.

Класс ConsoleHelper

Обеспечивает цветной вывод текста и оформление интерфейса.

Ключевой момент: Управление состоянием консоли

```
public static void WriteLineColored(string message, ConsoleColor color) { var originalColor = Console.ForegroundColor; // 1. Запомнили текущий цвет
Console.ForegroundColor = color; // 2. Установили новый Console.WriteLine(message); // 3. Вывели сообщение Console.ForegroundColor = originalColor; // 4. Вернули
исходный цвет }
```

Работает как «обертка» (wrapper). Это предотвращает ситуацию, когда после вывода ошибки красным цветом весь остальной интерфейс остается красным. Метод гарантирует возврат к стандартным настройкам.

Класс Game

Управляет игровым процессом, циклами ходов, переключением игроков и проверкой победы.

Ключевой момент: Безопасный ввод (Parsing)

Обработка ввода пользователя реализована через метод `TryParseMove`.

```
private bool TryParseMove(string input, out int x, out int y) { // ... return parts.Length == 2 && int.TryParse(parts[0], out x) && int.TryParse(parts[1], out y);
}
```

Использует `int.TryParse`, который не выбрасывает исключений при ошибке (например, если введены буквы), а просто возвращает `false`. Это делает приложение устойчивым к некорректным действиям пользователя.

Игровой цикл:

Метод `play` содержит бесконечный цикл `while(true)`, который управляет ходом игры до тех пор, пока не будет выявлен победитель или пользователь не выйдет в меню.

Класс Menu

Реализует главное меню, включая запуск новой игры, сохранение, загрузку, настройки и таблицу рекордов.

Ключевой момент: UX и форматирование

```
ConsoleHelper.WriteLineColored($"{\"Name\",-15}{\"Color\",-15}{\"Score\",-10}\", ConsoleColor.Green);
```

Использование интерполяции строк с параметром выравнивания {-15} позволяет выводить данные в виде аккуратной таблицы, где колонки ровные независимо от длины имени игрока.

Машина состояний:

Используется конструкция `switch (choice)` для переключения между режимами работы приложения (Игра, Настройки, Рекорды), что упрощает навигацию.

Класс Player

Описывает сущность игрока (имя, цвет, счёт).

Ключевой момент: Автосвойства (Auto-properties)

```
public string Name { get; set; } = null!; public ConsoleColor Color { get; set; }
```

Класс выполняет роль DTO (Data Transfer Object). Он инкапсулирует данные игрока, не содержит сложной логики, и служит контейнером для передачи информации между другими частями системы.

Класс SaveManager

Отвечает за сохранение и загрузку состояния игры и таблицы рекордов в формате JSON.

Ключевой момент: Конфигурация сериализатора

```
new JsonSerializerOptions { WriteIndented = true, Converters = { new TwoDimensionalIntArrayConverter() } }
```

При создании настроек для `JsonSerializer` мы явно подключаем кастомный конвертер. Это сообщает системе, как правильно обрабатывать сложные типы данных (матрицу игрового поля), которые стандартный сериализатор не поддерживает.

Класс TwoDimensionalIntArrayConverter

Обеспечивает сериализацию и десериализацию двумерного массива `int[,]` для корректного сохранения игрового поля.

Ключевой момент: Ручная запись структуры JSON

```
public override void Write(Utf8JsonWriter writer, int[,] value, ...) { writer.WriteStartArray(); // Пишем '[' for (int i = 0; i < rows; i++) { ... } // Проходим по строкам writer.WriteEndArray(); // Пишем ']' }
```

Метод трансформирует прямоугольный массив C# (`[,]`) в зубчатый массив JSON (`[[]]`), который является стандартом для текстовых форматов данных. Это решает проблему несовместимости типов.

Класс Program

Является точкой входа и запускает меню.

Ключевой момент: Top-level statements

```
using Gomoku; new Menu().Display();
```

Использование современных возможностей C# позволяет сократить шаблонный код (main, class Program). Вся инициализация сводится к созданию экземпляра меню и передаче ему управления.

Файл проекта (.csproj)

Задаёт параметры сборки и указывает целевую платформу .NET 9.0.

Ключевой момент: Целевая платформа

```
<TargetFramework>net9.0</TargetFramework>
```

Указывает компилятору использовать последнюю версию платформы .NET 9, что обеспечивает доступ к новым возможностям языка C# 12 и повышает производительность приложения.

Качество кода (Code Quality)

Проект выполнен в соответствии с критериями оценивания:

SRP (Single Responsibility)

Разделение логики, данных и UI.

Clean Code

Понятный нейминг, отсутствие магических чисел.

Warnings: 0

Код компилируется без предупреждений.

DRY (Don't Repeat Yourself)

Повторное использование кода (методы).

Заключение

В ходе работы игра Gomoku была успешно реализована.

- Выполнены все пункты ТЗ.
- Реализован надежный механизм сохранений.
- Код структурирован и расширяем.

Спасибо за внимание!